

DS_HW4_資訊三_嚴聲遠_111703009

程式碼運作邏輯設計：

利用腳本(.py)執行分別的演算法，腳本亦分別負責跑出折線圖。

詳細完整程式碼、成果請至：https://github.com/spaces-lalala/2024DS_HW

Hash Table 標準函式庫說明(unordered_map)：

1. 參考網站：

a. How To Implement a Sample Hash Table in C/C++

<https://www.digitalocean.com/community/tutorials/hash-table-in-c-plus-plus>

b. unordered_map in C++ STL

https://www.geeksforgeeks.org/unordered_map-in-cpp-stl/

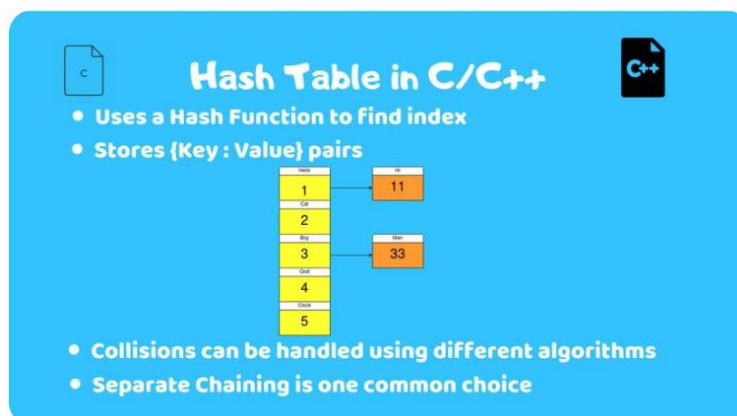
c. cplusplus.com std::unordered_map::find

https://cplusplus.com/reference/unordered_map/unordered_map/find/

d. cplusplus.com std::unordered_map::insert

https://cplusplus.com/reference/unordered_map/unordered_map/insert/

以下截圖來自以上網址：



`unordered_map<int, string> umap;`

Diagram illustrating the components of the `unordered_map` template:

- `int` is the **Key**.
- `string` is the **Value**.
- `umap` is the **Map name**.

std::unordered_map::insert

Complexity

Single element insertions:

Average case: constant.

Worst case: linear in container size.

->Average $O(1)$

std::unordered_map::find

Complexity

Average case: constant.

Worst case: linear in [container size](#).

->Average $O(1)$

Unordered_map 範例程式：

```
#include <iostream>
#include <unordered_map>
using namespace std;

// Driver code
int main()
{
    // Declaring umap to be of
    // <string, int> type key
    // will be of STRING type
    // and mapped VALUE will
    // be of int type
    unordered_map<string, int> umap;

    // inserting values by using [] operator
    umap["GeeksforGeeks"] = 10;
    umap["Practice"] = 20;
    umap["Contribute"] = 30;

    // Traversing an unordered map
    for (auto x : umap)
        cout << x.first << " " <<
            x.second << endl;
}
```

2. 說明：

<unordered_map> 是一種associated container，用於儲存由key value和mapped value組成的元素。key value用於唯一識別元素，而mapped value則是與key相關聯的內容。key和value的類型可以是預定義的，也可以是使用者自定義的。簡而言之，<unordered_map> 就像一種字典類型的資料結構，用於在內部儲存元素。它包含一系列的key value對 (key, value)，能夠根據其唯一的key快速檢索個別元素。

<unordered_map> 是透過Hash Table實現的，提供給mapped的key會被Hash處理成Hash Table中的索引。因此，這個資料結構的性能在很大程度上取決於Hash Function。不過，平均來說，在Hash表中搜尋、插入和刪除的操作成本為 $O(1)$ 。

Balanced BST 標準函式庫說明(map)：

1. 參考網站：

a.Self-Balancing Binary Search Trees

<https://www.geeksforgeeks.org/self-balancing-binary-search-trees/>

b.Cplusplus

<https://cplusplus.com/reference/map/map/insert/>

<https://cplusplus.com/reference/map/map/find/>

<https://learn.microsoft.com/zh-tw/troubleshoot/developer/visualstudio/cpp/libraries/use-map-stl-functions>

以下截圖來自以上網址：

std::map::insert

Complexity

If a single element is inserted, logarithmic in [size](#) in general, but amortized constant if a hint is given and the *position* given is the optimal.

->Amortized $O(1)$

std::map::find

Complexity

Logarithmic in [size](#).

->Amortized $O(\log(n))$

範例使用：

```
// 1. Create a map of ints to strings
INT2STRING theMap;
INT2STRING::iterator theIterator;
string theString = "";
int index;
// Fill it with the digits 0 - 9, each mapped to its string counterpart
// Note: value_type is a pair for maps...
theMap.insert(INT2STRING::value_type(0,"Zero"));
theMap.insert(INT2STRING::value_type(1,"One"));
theMap.insert(INT2STRING::value_type(2,"Two"));
theMap.insert(INT2STRING::value_type(3,"Three"));
theMap.insert(INT2STRING::value_type(4,"Four"));
theMap.insert(INT2STRING::value_type(5,"Five"));
theMap.insert(INT2STRING::value_type(6,"Six"));
theMap.insert(INT2STRING::value_type(7,"Seven"));
theMap.insert(INT2STRING::value_type(8,"Eight"));
theMap.insert(INT2STRING::value_type(9,"Nine"));
// Read a Number from the user and print it back as words
for( ; ; )
{
    cout << "Enter \"q\" to quit, or enter a Number: ";
    cin >> theString;
    if(theString == "q")
        break;
    // extract each digit from the string, find its corresponding
    // entry in the map (the word equivalent) and print it
    for(index = 0; index < theString.length(); index++){
        theIterator = theMap.find(theString[index] - '0');
        if(theIterator != theMap.end()) // is 0 - 9
            cout << (*theIterator).second << " ";
        else // some character other than 0 - 9
            cout << "[err] ";
    }
    cout << endl;
}
```

2. 說明：

C++ 中的 `map` 是一種基於 平衡二元搜尋樹（Balanced Binary Search Tree，通常為紅黑樹）實現的associated container，用於儲存key-value pairs。其特性包括：key的唯一性、key value自動排序、高效的插入、搜尋和刪除操作，時間複雜度為 $O(\log n)$ 。`map` 的底層實作使用平衡 BST，確保在多次插入與刪除後仍能保持樹的平衡性，使得所有操作的時間複雜度穩定且高效。每次插入時，紅黑樹通過旋轉和變色操作維持平衡；搜尋操作則依靠二元搜尋特性，能快速定位目標節點。在應用中，`map` 適合需要按鍵順序儲存資料並進行範圍查詢的場合，例如實現排序的字典、統計資料分布等。

Insert：

透過插入新元素來擴展容器，實際上會將容器的大小增加插入元素的數量。

由於 map 中的鍵 (key) 是唯一的，因此插入操作會檢查每個插入的元素，其鍵是否與容器中已存在的元素的鍵相同。如果相同，則該元素不會被插入，並返回一個指向已存在元素的迭代器（如果函數有返回值）。

Find：

搜尋容器中是否存在與鍵 k 相等的元素，若找到則返回指向該元素的迭代器，否則返回指向 map::end 的迭代器。如果容器的比較物件對兩個鍵進行比較時，無論元素的傳入順序如何都返回 false，則這兩個鍵被視為相等。

程式詳細說明：

1. 隨機生成插入和搜尋data vector：

```
vector<long long> data(n);
for (long long i = 0; i < n; i++) {
    data[i] = dist(gen);
}

vector<long long> search_data(100000);
for (int i = 0; i < 100000; i++) {
    search_data[i] = dist(gen);
}
```

利用隨機性質生成插入Data與搜尋Data。

2. Hash Table(Unorderedmap) 插入：

```
unordered_map<long long, int> hash_table;

for (long long i = 0; i < n; i++) {
    hash_table.insert({data[i], i});
}
```

利用Unorderedmap insert函數，利用已生成好的data陣列插入。

3. Hash Table(Unorderedmap) 搜尋：

```
for (int i = 0; i < 100000; i++) {
    hash_table.find(search_data[i]);
}
```

利用Unorderedmap find函數，利用已生成好的search data陣列搜尋。

4. Balanced BST(Map) 插入：

```
unordered_map<long long, int> hash_table;
```

```
for (long long i = 0; i < n; i++) {
    hash_table.insert({data[i], i});
}
```

利用Map insert函數，利用已生成好的data陣列插入。

5. Balanced BST(Map) 搜尋：

```
for (int i = 0; i < 100000; i++) {
    bst.find(search_data[i]);
}
```

利用Map insert函數，利用已生成好的data陣列搜尋。

6. 腳本執行(python)：

a. CPP執行：

```
def main():
    cpp_file = "hash_bst.cpp"
    executable = "./hash_bst.exe"
    subprocess.run(["g++", "-O3", cpp_file, "-o", executable])

    k_values = range(10, 31)
    results_df = run_experiment(executable, k_values)
    results_df.to_csv('results.csv', index=False)
    plot_results(results_df, 'output')
```

以腳本的方式執行cpp檔進行實驗測試時間。

b. 跑實驗：

```
def run_experiment(executable_path, k_values):
    results = []
    columns = ['k', 'n', 'hash_insert_time', 'hash_search_time', 'bst_insert_time', 'bst_search_time']

    for k in k_values:
        print(f"Running experiment for k = {k}")
        n = 2**k

        if k > 24:
            if results:
                last_df = pd.DataFrame([results[-1]], columns=columns).iloc[0]
                predicted = predict_times(last_df, k)
                results.append([
                    k,
                    predicted['n'],
                    predicted['hash_insert_time'],
                    predicted['hash_search_time'],
                    predicted['bst_insert_time'],
                    predicted['bst_search_time']
                ])
                print(f"Added predicted results for k = {k}")
                continue

        try:
            result = subprocess.run([executable_path, str(k)],
                                    capture_output=True,
                                    text=True,
                                    timeout=600)
            if result.returncode == 0:
                output = list(map(float, result.stdout.strip().split(',')))
                results.append([k, n, output[1], output[2], output[3], output[4]])

        except subprocess.TimeoutExpired:
            print(f"Timeout for k = {k}")
            if results:
                last_df = pd.DataFrame([results[-1]], columns=columns).iloc[0]
                predicted = predict_times(last_df, k)
                results.append([
                    k,
                    predicted['n'],
                    predicted['hash_insert_time'],
                    predicted['hash_search_time'],
                    predicted['bst_insert_time'],
                    predicted['bst_search_time']
                ])
            continue

    return pd.DataFrame(results, columns=columns)
```

主要執行的程式內容，由於超出 2^{24} 的時候會超出記憶體無法正常執行，故後面直接執行預測函數預測缺失資料。實驗以及預測完資料點後再進行畫圖。

c. 預測缺失資料

```
def predict_times(last_results, k):
    """Predict times for both insertion and search operations based on theoretical complexity."""
    n_old = float(last_results['n'])
    n_new = 2**k
    n_ratio = n_new / n_old

    hash_insert_growth = n_ratio # O(n)
    bst_insert_growth = n_ratio * (np.log2(n_new) / np.log2(n_old)) # O(n log n)

    hash_search_growth = 1 + np.log2(n_ratio) * 0.1 # O(1) with slight degradation
    bst_search_growth = np.log2(n_new) / np.log2(n_old) # O(log n)

    return {
        'k': k,
        'n': n_new,
        'hash_insert_time': min(600, round(float(last_results['hash_insert_time']) * hash_insert_growth, 3)),
        'hash_search_time': min(600, round(float(last_results['hash_search_time']) * hash_search_growth, 3)),
        'bst_insert_time': min(600, round(float(last_results['bst_insert_time']) * bst_insert_growth, 3)),
        'bst_search_time': min(600, round(float(last_results['bst_search_time']) * bst_search_growth, 3))
    }
```

Hash Table Insertion (hash_insert_growth) :

Hash table 的插入操作在理論上具有線性時間複雜度 $O(n)$ ，因為插入所有元素需要遍歷輸入數據的大小 n 。在程式中，利用新舊數據大小比例 $n_ratio = n_new / n_old$ 預測插入時間。此方法假設在數據量增大時，插入操作的時間直接與數據量成正比。

BST Insertion (bst_insert_growth) :

二元搜索樹 (BST) 的插入操作具有 $O(n \log(n))$ 的理論時間複雜度。這是因為每個插入操作需要遍歷樹的高度約為 $(\log(n))$ ，並插入 n 個元素。公式中使用了對數的比例來反映數據增長對插入時間的影響：

$$bst_insert_growth = n_ratio \times \log_2(n_new) / \log_2(n_old)$$
 這個增長公式捕捉了隨著數據增長，搜索樹高度變化對插入操作的影響。

Hash Table Search (hash_search_growth) :

Hash table 的搜尋操作理論上是常數時間 $O(1)$ ，但隨著數據量增大，可能會因 Hash table 碰撞導致效率略微下降。這裡的公式通過增加一個與 $\log_2(n_ratio)$ 成比例的微小項，模擬可能的效率降低情況：

$$hash_search_growth = 1 + 0.1 \times \log_2(n_ratio)$$
 該公式假設 Hash Table 的搜尋性能受數據量影響很小，但仍會隨數據量增大略有劣化。

BST Search (bst_search_growth)

BST 搜尋操作的理論時間複雜度為 $O(\log n)$ ，因為每次搜索需要遍歷樹的高度，這與數據量的對數成正比。在程式中，使用對數比例預測搜尋時間： $\text{bst_search_growth} = \log_2(n_{\text{new}})/\log_2(n_{\text{old}})$ ，此公式準確反映了隨著數據增長，BST 高度增加對搜尋操作時間的影響。

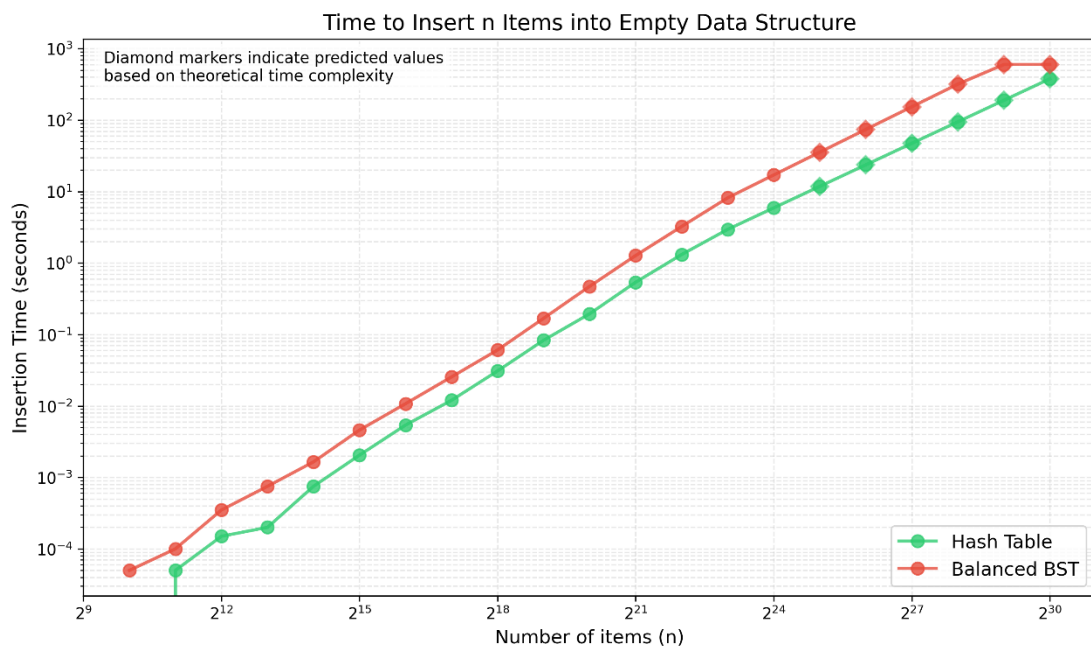
實驗圖分析：

詳細折線圖與數據同前文連結：https://github.com/spaces-alala/2024DS_HW

由於 RAM 問題，以下菱形資料點是預測的

1. 第一張折線圖：

折線圖：



Hash Table 2^{10} 測量數據由於精度不足故顯示 0。

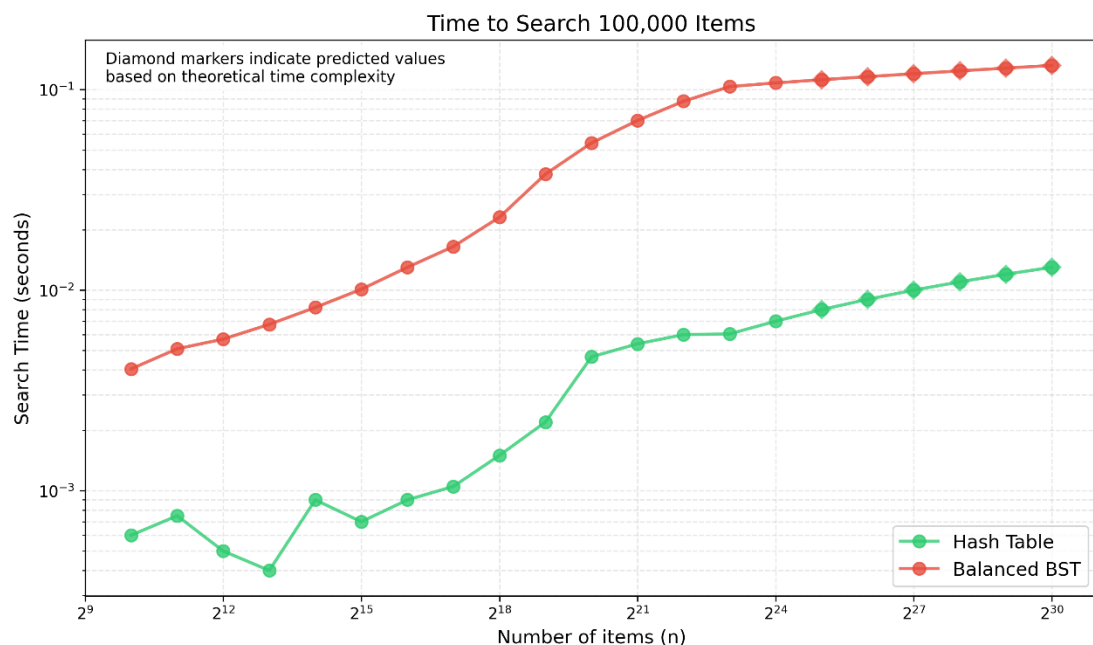
分析：

理論上 Hash Table 插入操作的理論時間複雜度為 $O(1)$ 平均情況下，其插入 n 個元素的總時間複雜度為 $O(n)$ 。每次插入時，Hash Table 通過計算 Hash Function 直接定位存儲位置，因此其增長率是線性的，與插入的資料量 n 成

正比。而觀察實驗結果，插入時間與理論增長趨勢一致，表明其插入效率在整體上保持穩定，主要受資料量線性增長。

Balanced BST: 插入操作的理論時間複雜度為 $O(\log(n))$ ，插入 n 筆資料的總時間複雜度為 $O(n\log(n))$ 。這是因為每次插入時需要遍歷樹的高度，而高度與樹中節點的數量呈對數關係。因此，BST 的插入時間會比 Hash Table 隨著資料量增大而更快上升。觀察實驗結果，插入時間隨資料量增長，符合 $O(n\log(n))$ 的理論增長趨勢，相較 Hash Table 所需時間更多。

2. 第二張折線圖：



Hash Table 與平衡二元搜尋樹 (Balanced Binary Search Tree, BST) 在搜尋上的理論效能有所不同。Hash Table 的搜尋平均時間複雜度 $O(1)$ ，但在最壞情況下可能達到 $O(n)$ (當碰撞過多時)。平衡 BST 的搜尋時間複雜度為 $O(\log(n))$ ，因為其深度與節點數的對數成正比。在這次實驗中，折線圖顯示，當資料量 n 增加時，Hash Table 的搜尋時間幾乎保持穩定 (略有波動)，而平衡 BST 的搜尋時間則呈現指數型增長，符合理論預期。這反映了 Hash Table 在大數據量時的高效特性，特別是在碰撞機制有效的情況下。

從實驗結果中可以看出，平衡 BST 的搜尋時間明顯高於 Hash table，這主要是由於其結構特性和操作流程造成的。平衡 BST 在搜尋時需要從根節點依序比對，並通過多層節點逐步接近目標資料，因此隨著資料量增大，搜尋路徑的長度（即樹的高度）增加，導致耗時明顯。而 Hash Table 透過 Hash function 直接計算出目標資料的位置，理論上能避免大部分不必要的遍歷。實驗結果表明，隨著 n 增大，平衡 BST 的搜尋時間比 Hash Table 以更快的速度增長，這與理論的時間複雜度分析完全一致。此外，當資料量超大時，平衡 BST 的效能會顯得更為劣勢，這也是其本質結構限制的體現。

問題&心得：

1. Hash Table 在 k 還比較小時搜尋的時間有震盪的趨勢，原先不確定是否是由於碰撞而導致的誤差，有嘗試增加實驗次數，震盪有變小但依舊有明顯的震盪狀況。
2. 原先預期在用完 RAM 時(2^{25})會進入使用 VRAM 而導致時間不符合理論，但在實驗時，無法成功使用 VRAM，故也無法回傳在 2^{25} 以上的數據，僅能使用預測的方式。
3. 相比於前幾次的作業，使用標準函式庫實作上能夠自動地避免很多 Bug，故使用上的安全性較高，且使用上也較簡單，有很多使用上的教學與範例可使用，複雜度上也經過一定的檢視與測試。